Numerisk matematik – at regne med computer

De fleste ligninger i fysik og teknik kan ikke løses eksakt. Numerisk matematik handler om at finde tilnærmede løsninger – og om at vide, hvor gode de er. I 1970'erne regnede man typisk på universitetets centrale computer og på de første programmerbare lommeregnere; i dag har enhver telefon mere regnekraft end alle Sussex' computere tilsammen dengang. Metoderne er de samme. Siden har en interaktiv sammenligning af metoder til ligningsløsning og til differentialligninger.

> University of Sussex, Numerical Mathematics (1975–78): Et af matematikkurserne i fysikstudiets pensum. Ti år senere skulle jeg bruge Newton–Raphsons og Mullers metoder til at finde komplekse rødder i mit eksamensarbejde på DTH. Siden beskriver emnet, som det normalt undervises; den er ikke en gengivelse af det oprindelige kursus.

Afrundingsfejl

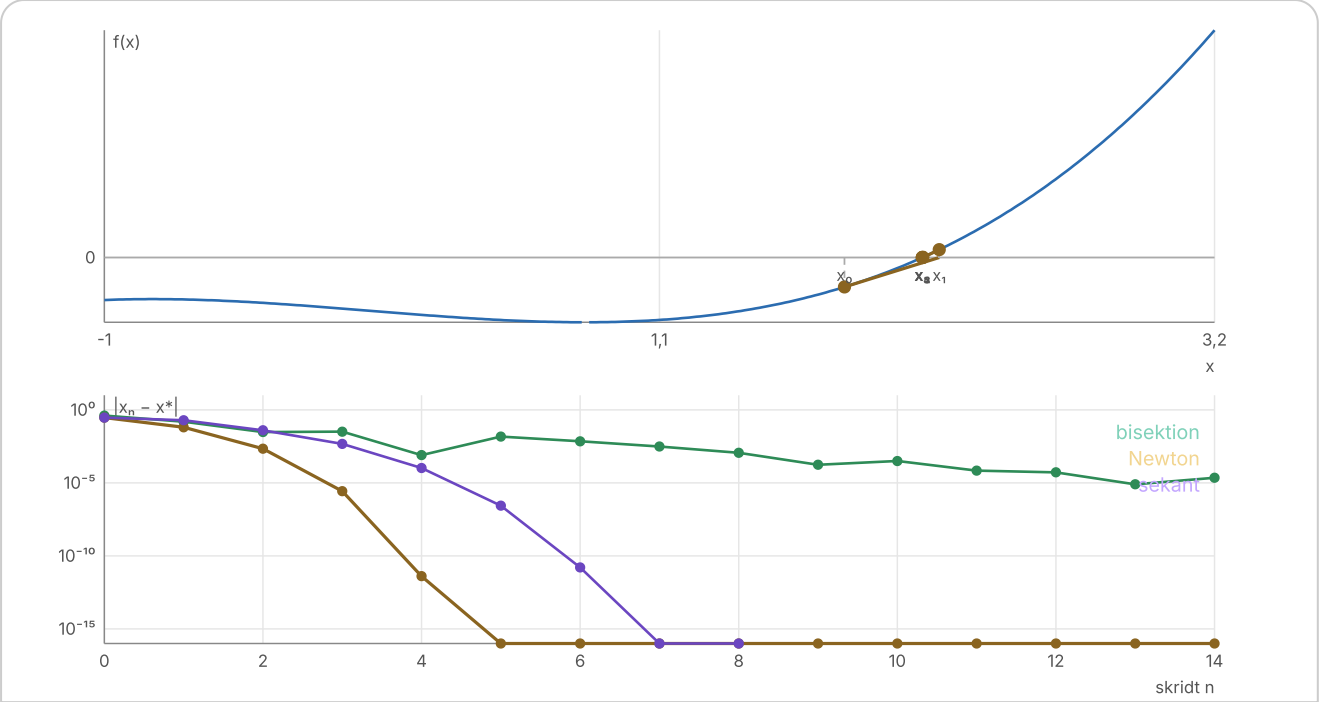
En computer regner med et endeligt antal cifre. Flydende tal i dobbelt præcision (IEEE 754, standardiseret i 1985) har 53 binære cifre, omkring 16 decimaler: maskinpræcisionen er $\varepsilon \approx 2{,}2 \cdot 10^{-16}$. Det lyder af meget, men fejlene kan vokse. Trækker man to næsten lige store tal fra hinanden, forsvinder de forreste cifre, og den relative fejl eksploderer – *katastrofal udslukning*. Den kendte formel for andengradsligningens rødder giver derfor dårlige resultater, når $b^2 \gg 4ac$, og man må omskrive den. Det andet store fejlbidrag er *trunkeringsfejlen*: når en uendelig proces (en række, en grænseværdi) afbrydes efter endelig mange skridt. Kunsten er at balancere de to.

Ligningsløsning

For at løse $f(x) = 0$ findes en række iterative metoder:

- **Bisektion:** Kender man et interval, hvor f skifter fortegn, halveres det igen og igen. Metoden er langsom – én binær decimal pr. skridt – men den svigter aldrig.
- **Newton–Raphson:** Tangenten i det aktuelle punkt følges ned til x-aksen: $x_{n+1} = x_n - f(x_n)/f'(x_n)$. Nær en simpel rod er konvergensen *kvadratisk*: antallet af korrekte cifre fordobles ved hvert skridt. Men et dårligt startgæt kan sende den langt væk, og ved en dobbeltrod bliver den langsom.
- **Sekantmetoden:** Som Newton, men med en sekant gennem de to seneste punkter i stedet for tangenten – ingen afledt nødvendig, og konvergensordenen er det gyldne snit, 1,618.
- **Mullers metode** (1956) lægger en parabel gennem tre punkter og kan finde komplekse rødder, selv fra reelle startværdier.

Interaktiv rodsøgning



Interpolation

Kendes en funktion kun i nogle punkter, kan man lægge et polynomium gennem dem. Med $n+1$ punkter findes netop ét polynomium af grad n (Lagrange- eller Newton-form). Men høje grader er farlige: Carl Runge viste i 1901, at interpolation af $1/(1+25x^2)$ i ækvidistante punkter svinger voldsomt ved enderne, og at det bliver værre, jo flere punkter man tager. Løsningen er enten at vælge punkterne tættere ved enderne (Chebyshev-punkter) eller at bruge stykvise polynomier – kubiske splines, der var opkaldt efter de bøjelige lister, skibsbyggere brugte til at tegne skrog.

Numerisk integration

Et integral tilnærmes med en vægtet sum af funktionsværdier. Trapezreglen erstatter funktionen med rette linjestykker og har en fejl, der falder som h^2, hvor h er skridtlængden. Simpsons regel bruger parabler gennem tre punkter og har fejlen h^4 – halveres skridtet, bliver fejlen 16 gange mindre. Gauss-kvadratur vælger både punkter og vægte optimalt og integrerer polynomier af grad $2n-1$ eksakt med n punkter. Richardson-ekstrapolation kombinerer resultater med forskellig h for at fjerne de ledende fejlled – det er grundlaget for Romberg-integration.

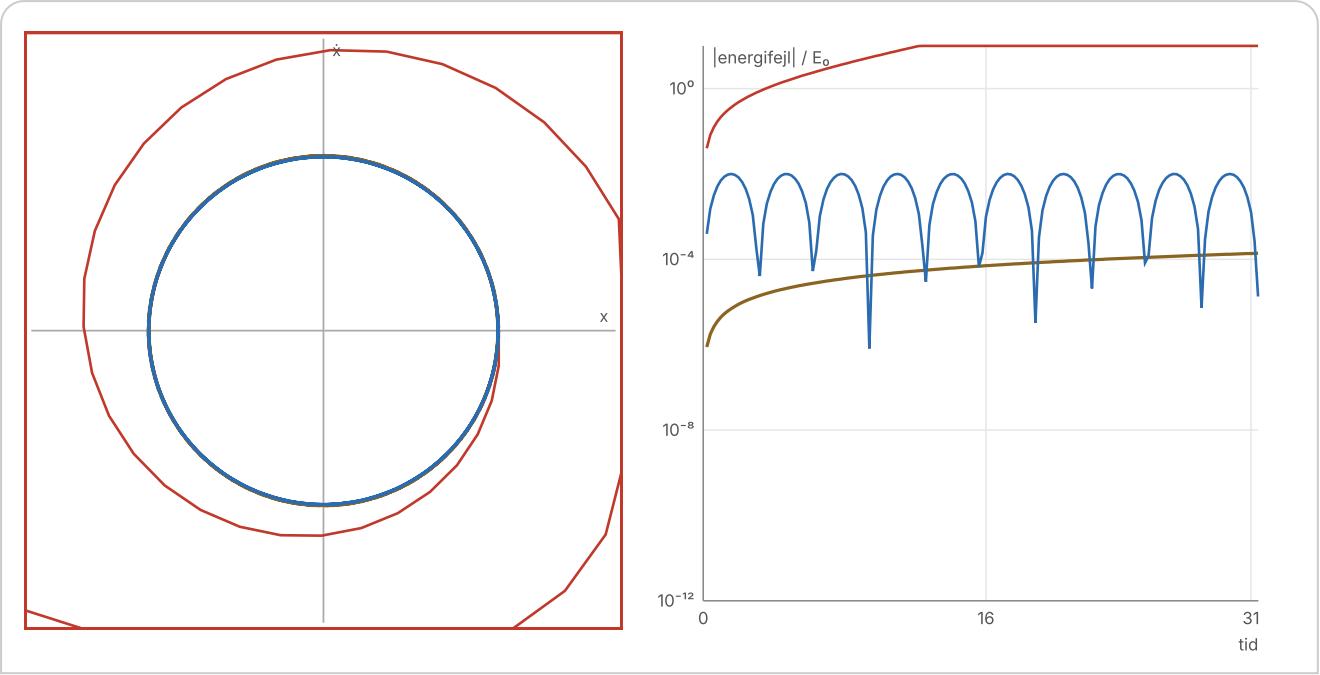
Lineære ligningssystemer

Gauss-elimination løser $A\vec{x}=\vec{b}$ med omkring $\tfrac{2}{3}n^3$ operationer. For at undgå at dividere med små tal bytter man rækker, så det største element står i diagonalen – *partiel pivotering*. Konditionstallet $\kappa(A)=\|A\|\,\|A^{-1}\|$ fortæller, hvor meget fejl i data forstærkes: med $\kappa=10^{10}$ mister man ti af de

seksten cifre. Store, tynde systemer fra fx diskretiserede differentialligninger løses bedre iterativt, med Jacobi, Gauss–Seidel eller SOR (se relaksationen i anvendt feltteori).

Differentialligninger

En differentialligning $\dot{y} = f(t, y)$ løses skridt for skridt. Eulers metode $y_{n+1} = y_n + h f(t_n, y_n)$ følger tangenten og har en global fejl proportional med h. Runge–Kuttas metode af fjerde orden (Runge 1895, Kutta 1901) vurderer hældningen fire gange pr. skridt og har fejlen h^4 – det er arbejdshesten i al videnskabelig regning. For mekaniske systemer er det ofte vigtigere at bevare energien end at være præcis på kort sigt: *symplektiske* metoder som Störmer–Verlet (leapfrog) holder energifejlen begrænset i al evighed, mens Euler lader den vokse og RK4 lader den drive langsomt.



Siden 1978

- **IEEE 754** (1985) gjorde flydende tal ens på alle computere, så et program giver samme resultat overalt.
- **Biblioteker:** LINPACK og EISPACK fra 1970'erne blev til LAPACK (1992), der i dag ligger under MATLAB, NumPy og næsten al videnskabelig software.
- **Adaptive løsere:** Moderne ODE-løsere (Dormand–Prince 1980, bag MATLAB's ode45) vælger selv skridtlængden ud fra et fejlestimat.
- **Grafikprocessorer** har gjort parallelle beregninger billige – fra klimamodeller til de neurale netværk, der trænes med varianter af gradientmetoden.
- **Fejlanalyse** har fået ny betydning med maskinlæring, hvor man ofte regner bevidst med lav præcision (16 eller 8 bit) for at spare tid og energi.

Læs mere

- William H. Press m.fl.: *Numerical Recipes* (3. udg., Cambridge 2007)
- Lloyd N. Trefethen og David Bau: *Numerical Linear Algebra* (SIAM 1997)
- Ernst Hairer, Christian Lubich og Gerhard Wanner: *Geometric Numerical Integration* (2. udg., Springer 2006)